

Deep Learning, DL Models, and Applications

Skill Development Tasks (1 to 15)

Task 1: Vectorized Multi-Layer Perceptron (MLP) Forward & Backward Pass from Scratch

- **Objective:** Master the mathematical foundations of gradient descent by deriving and implementing multi-layer backpropagation equations using raw matrix calculus from first principles.
- **Required Tech Stack:** Pure NumPy, Python 3.10+, Jupyter Notebook.
- **Task Description:** Students must implement a fully connected neural network with arbitrary hidden layer dimensions from scratch. The code must handle batch-wise forward passes, calculate cross-entropy loss, derive layer-wise local Jacobians, and execute backward weight adjustments using raw 3D matrix transformations without PyTorch's autograd.

Task 2: Empirical Diagnostic of Custom Weight Initializations and Gradient Vanishing

- **Objective:** Solve mathematical training instability in deep networks by analyzing activation distributions and gradient magnitudes under varying initialization distributions.
- **Required Tech Stack:** PyTorch (with Autograd explicitly disabled via `requires_grad=False`), NumPy, Matplotlib.
- **Task Description:** Students will design custom Xavier (Glorot) and Kaiming (He) weight initialization routines. They must write a tracking harness that hooks into an uninitialized 20-layer MLP to log and plot the mean and variance of intermediate activation vectors and weight gradients across epochs, demonstrating gradient vanishing or explosion.

Task 3: Vectorized Convolutional Forward Pass via im2col Transformation from Scratch

- **Objective:** Understand the mechanics of spatial feature extractors by implementing vectorized convolutional layers optimized for high-performance matrix multiplications.
- **Required Tech Stack:** Pure NumPy, Python 3.10+, SciPy.
- **Task Description:** Students must implement a 2D convolutional layer supporting arbitrary padding, strides, and input/output channels from scratch. They are required to write an `im2col` matrix helper to reshape the input tensor into a unified 2D matrix, transforming sliding window convolutions into a single dot-product operation (

$$C_{out} \times (C_{in} \cdot K_h \cdot K_w)).$$

Task 4: Custom Gradient Descent Optimizers with Momentum, RMSprop, and Adam

- **Objective:** Master adaptive learning rate mechanics and temporal update optimization algorithms by implementing high-performance weight updates from scratch.
- **Required Tech Stack:** PyTorch (using only tensor mathematics, no torch.optim), NumPy.
- **Task Description:** Students will design a training harness utilizing PyTorch tensors. They must write optimizer update rules from scratch, calculating exponential moving averages of first and second raw moments, applying bias corrections, and tracking loss convergence rates on non-linear datasets.

Task 5: Custom Batch Normalization and Layer Normalization Forward/Backward Propagation

- **Objective:** Maximize training convergence speeds and stabilize internal covariate shifts by building structural normalization layers with manual gradient derivations.
- **Required Tech Stack:** PyTorch (with Autograd disabled), NumPy.
- **Task Description:** Students must write functional modules for Batch Normalization and Layer Normalization. They are required to derive and implement the forward passes (calculating running mean and variance) and write the exact partial derivatives for backpropagating gradients through the scale parameter γ , shift parameter β , and normalize transformations.

Task 6: Bidirectional Long Short-Term Memory (BiLSTM) Cell Mechanics from Scratch

- **Objective:** Learn sequence transduction architectures and dynamic temporal tracking by implementing gated recurrent equations and memory routing layouts from scratch.
- **Required Tech Stack:** PyTorch (Tensor operations only, no nn.RNN or nn.LSTM), NumPy.
- **Task Description:** Students will design and execute a bidirectional LSTM layer. They must write individual loop-based step updates calculating Input (i_t), Forget (f_t), Output (o_t), and Cell State Candidate (g_t) activations, stitch backward and forward sequence steps, and handle dynamic sequence-padded packs.

Task 7: ResNet-50 Style Residual Bottleneck Block and Grouped Convolutions

- **Objective:** Master deep architectural scaling paradigms by constructing bottleneck projection networks and multi-channel grouped/depthwise separable filters.
- **Required Tech Stack:** PyTorch, TorchVision, PyTorch Profiler.
- **Task Description:** Students must build a custom ResNet-50 bottleneck block featuring depthwise separable convolutions and learnable skip projection layers. They will run

PyTorch Profiler to analyze and optimize memory footprints, parameter counts, and floating-point execution steps (FLOPs) under different channel scaling factors.

Task 8: Semantic Segmentation Pipeline on Pixel-wise Grids using custom Dice Loss

- **Objective:** Construct high-precision spatial localization engines for medical imaging and autonomous navigation datasets, utilizing advanced boundary overlap losses.
- **Required Tech Stack:** PyTorch, TorchVision, Albumentations, Matplotlib.
- **Task Description:** Build a deep U-Net architecture for segmenting complex image masks. Students must implement a custom composite loss function combining Soft Dice Loss and Binary Cross-Entropy, write dataset pipelines parsing large spatial target masks, and plot pixel-wise precision-recall boundary curves.

Task 9: Anchor-Free Object Detection (YOLOv8-style) Loss Backpropagation

- **Objective:** Architect state-of-the-art multi-task object localization pipelines by tracking regression outputs and classification predictions simultaneously.
- **Required Tech Stack:** PyTorch, OpenCV, NumPy, Matplotlib.
- **Task Description:** Students will write a custom loss-tracking module for an anchor-free object detector. The module must implement Complete Intersection over Union (CIoU) bounding box regression loss, compute classification loss, and handle complex assignment strategies to match target labels with dynamic network outputs.

Task 10: Wasserstein GAN with Gradient Penalty (WGAN-GP) for Stable Image Synthesis

- **Objective:** Stabilize adversarial generative training networks by implementing Wasserstein distance metrics with strict Lipschitz-1 gradient constraints.
- **Required Tech Stack:** PyTorch, TorchVision, Matplotlib.
- **Task Description:** Students must implement a generative adversarial model with WGAN-GP. They are required to design generator and critic architectures, write custom mathematical equations to calculate the gradient penalty loss $(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2$ on interpolated inputs, and analyze training logs over 100 epochs.

Task 11: Vision Transformer (ViT) Patch Projection and Multi-Head Self-Attention from Scratch

- **Objective:** Adapt Transformer architectures to visual data by constructing spatial patch extraction modules and multi-head representation attention matrices.
- **Required Tech Stack:** PyTorch, Einops, Transformers.
- **Task Description:** Students will build a Vision Transformer block from first principles. They must write a custom linear patch projection layer, append learnable class and positional

embeddings, implement multi-head self-attention using Einstein summation syntax via einops, and visualize attention maps on high-resolution images.

Task 12: High-Dimensional Latent Space Anomaly Detection via Convolutional Autoencoders

- **Objective:** Identify micro-structural patterns and outliers in manufacturing defect datasets by mapping reconstruction error fields within structural bottlenecks.
- **Required Tech Stack:** PyTorch, SciPy, Scikit-Learn, Matplotlib.
- **Task Description:** Build a Convolutional Autoencoder featuring dense structural bottleneck layers. Students must train the model exclusively on fault-free images, formulate custom pixel-wise structural similarity index (SSIM) loss functions, and design an inference loop that maps reconstruction error distributions to locate anomalies on test images.

Task 13: Temporal Convolutional Networks (TCN) with Dilated Causal Convolutions

- **Objective:** Model complex, non-linear multi-dimensional time-series sequences by constructing receptive field architectures with zero temporal look-ahead leaks.
- **Required Tech Stack:** PyTorch, Pandas, NumPy, Matplotlib.
- **Task Description:** Students must build a Temporal Convolutional Network. They must write customized 1D convolutional layers featuring dilated causal convolutions, systematically doubling dilation factors ($d \in [1, 2, 4, 8, 16]$) to capture long-range historical structures, and evaluate predictive errors against classical sequential models.

Task 14: Custom CUDA-Accelerated Swish Activation Layer Integration

- **Objective:** Bridge the gap between Python-level scriptability and raw GPU hardware execution by building custom CUDA kernels to accelerate non-linear operations.
- **Required Tech Stack:** C++, CUDA Toolkit, PyTorch C++ Extensions (cpp_extension).
- **Task Description:** Students must write a C++ header and CUDA kernel (`0.cu` file) to execute a thread-blocked, accelerated Swish activation function $f(x) = x \cdot \sigma(\beta x)$ on high-dimensional tensors. They will compile this module using PyTorch's compiler bindings and benchmark its runtime efficiency against standard framework layers.

Task 15: Distributed Data Parallel (DDP) Multi-GPU Node Sync Gateway with GLOO/NCCL

- **Objective:** Master large-scale deep learning deployment strategies by orchestrating distributed training runs synced across isolated hardware nodes.
- **Required Tech Stack:** PyTorch Distributed, Docker-Compose, NCCL Toolkit.
- **Task Description:** Students will construct a containerized distributed training loop. They

must configure dynamic model partitioning using PyTorch DDP, deploy a multi-container Docker cluster simulating independent training nodes, manage state synchronizations using GLOO/NCCL backends, and benchmark parameter synchronization scaling metrics.